

1. Bộ nhớ cấp phát động lấy từ vùng nào trong RAM?

Bộ nhớ cấp phát động được lấy từ vùng **Heap**. Khác với vùng Stack (dành cho biến cục bộ), vùng Heap là một khu vực bộ nhớ lớn và tự do, cho phép chương trình yêu cầu và giải phóng bộ nhớ một cách linh hoạt tại thời điểm thực thi (runtime).

2. Điểm khác biệt lớn nhất giữa Stack và Heap là gì?

Điểm khác biệt cốt lõi nằm ở **cách quản lý và thời gian tồn tại**:

- **Stack**: Việc cấp phát và giải phóng được thực hiện **tự động** bởi trình biên dịch khi vào và thoát khỏi hàm.
- **Heap**: Việc cấp phát và giải phóng hoàn toàn do **lập trình viên kiểm soát**. Nếu bạn cấp phát mà quên giải phóng (free), sẽ dẫn đến hiện tượng rò rỉ bộ nhớ (memory leak).

3. Các hàm chính dùng để quản lý bộ nhớ trong C

Trong thư viện tiêu chuẩn <stdlib.h>, có 4 hàm cơ bản:

- malloc()
- calloc()
- realloc()
- free()

4. So sánh hàm calloc() và malloc()

Mặc dù cả hai đều dùng để xin cấp phát bộ nhớ, chúng có hai sự khác biệt chính:

Đặc điểm	malloc()	calloc()
Giá trị khởi tạo	Chứa giá trị rác (không khởi tạo).	Tự động xóa bộ nhớ và khởi tạo tất cả các bit bằng 0 .
Tham số truyền vào	1 tham số: Tổng số byte cần xin.	2 tham số: Số lượng phần tử và kích thước mỗi phần tử.

5. Hàm realloc() dùng để làm gì?

Hàm realloc() (viết tắt của *re-allocate*) được dùng để **thay đổi kích thước** của một khối bộ nhớ đã được cấp phát trước đó bởi malloc() hoặc calloc().

- Nếu vùng nhớ hiện tại còn trống ở phía sau, nó sẽ mở rộng thêm.
- Nếu không đủ chỗ, nó sẽ tìm một vùng nhớ mới đủ lớn, sao chép dữ liệu cũ sang và giải phóng vùng nhớ cũ.

6. Có được free() một biến cục bộ không?

Câu trả lời là: **KHÔNG**.

- **Tại sao?** Hàm `free()` chỉ được sử dụng để giải phóng bộ nhớ được cấp phát động trên **Heap** (thông qua `malloc`, `calloc`, `realloc`).
- **Hậu quả:** Biến cục bộ (ví dụ `int x`) nằm trên **Stack**. Khi bạn cố gắng gọi `free(&x)`, chương trình sẽ gặp lỗi **Runtime Error** (thường là lỗi "invalid pointer" hoặc "segmentation fault") vì trình quản lý bộ nhớ không tìm thấy địa chỉ đó trong danh sách quản lý của Heap.

Nguồn tham khảo chính:

1. Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
2. GNU C Library Documentation - Memory Allocation.
3. C Standard Library (ISO/IEC 9899) - Section 7.22.3 Memory management functions.